Lec27

API

Application Programming Interface

An API is a bridge that allows two software applications to communicate with each other.

Think of an API as a **waiter in a restaurant**.

Customer
↓
Waiter (API)
↓
Kitchen (Server)
↓
Waiter (API)
↓
Customer

What is REST?

REST means

Representational State Transfer

REST is a set of rules used for designing web APIs.

A REST API uses

- HTTP methods
- URLs
- Resources
- Status Codes

Resources

Everything in REST is called a **Resource**.

Examples

Student

Teacher

Product

Order

Employee

Customer

RESTful API

A RESTful API follows REST principles.

Instead of creating URLs like

/getStudents

/addStudent

/deleteStudent

/updateStudent

REST Architecture

Client

↓

HTTP Request

↓

Express Server

↓

Controller

↓

Database

↓

Response

HTTP Methods

Method	Purpose
GET	Read data
POST	Create data
PUT	Replace data
PATCH	Update some fields
DELETE	Remove data

Sample Data

```
let students = [
   { id: 1, name: "Rahul", age: 20 },
   { id: 2, name: "Amit", age: 21 }
];
```

GET Method

Purpose

Retrieve data.

GET never creates data.

GET never deletes data.

GET never updates data.

It only reads.

Get all Students

```
const express = require("express");

const app = express();

app.use(express.json());

let students = [
    { id: 1, name: "Rahul", age: 20 },
    { id: 2, name: "Amit", age: 21 }
];

app.get("/students", (req, res) => {

    res.json(students);

});

app.listen(3000);
```

Get Single Student

```
app.get("/students/:id", (req, res) => {

    const id = parseInt(req.params.id);

    const student = students.find(s => s.id === id);

    if (!student) {
        return res.status(404).json({
            message: "Student Not Found"
        });
    }

    res.json(student);

});
```

POST Method

POST creates new data.

Real Example

Facebook

↓

Create Account

↓

POST Request

↓

User Added

Add Student

```
app.post("/students", (req, res) => {
    const newStudent = {
        id: students.length + 1,
        name: req.body.name,
        age: req.body.age
    };
    students.push(newStudent);
    res.status(201).json({
        message: "Student Added",
        student: newStudent
    });
});
```

PUT Method

PUT replaces the **entire resource**.

Imagine a student record

Rahul

Age 20

City Delhi

PUT replaces everything.

New Data

Rahul Sharma

Age 21

City Jaipur

Old record disappears.

Example

```
app.put("/students/:id", (req, res) => {

const id = parseInt(req.params.id);

const index = students.findIndex(s => s.id === id);

if (index === -1) {

return res.status(404).json({
message: "Student Not Found"
});

}
```

```
students[index] = {
id,
name: req.body.name,
age: req.body.age
};
res.json({
message: "Student Updated",
student: students[index]
});
});
```

PATCH Method

PATCH updates **only specific fields**.

Suppose

Rahul

20

Delhi

Only age changes.

Rahul

21

Delhi

Everything else remains unchanged.

Example

```
app.patch("/students/:id", (req, res) => {

const id = parseInt(req.params.id);

const student = students.find(s => s.id === id);

if (!student) {

return res.status(404).json({
message: "Student Not Found"
});

}
```

```
if (req.body.name !== undefined) {
student.name = req.body.name;
}

if (req.body.age !== undefined) {
student.age = req.body.age;
}

res.json({

message: "Student Partially Updated",

student

});

});
```

DELETE Method

DELETE removes data permanently.

Example

```
app.delete("/students/:id", (req, res) => {

const id = parseInt(req.params.id);

const index = students.findIndex(s => s.id === id);

if (index === -1) {

return res.status(404).json({
message: "Student Not Found"
});

}

const deletedStudent = students.splice(index, 1);

res.json({

message: "Student Deleted",

student: deletedStudent[0]

});

});
```

Difference between PUT and PATCH

Feature	PUT	PATCH
Updates	Entire resource	Selected fields
Missing fields	Replaced/removed	Remain unchanged
Request body	Full object	Partial object
Use case	Replace an existing record	Edit one or more fields

Complete CRUD API

```
const express = require("express");

const app = express();

app.use(express.json());

let students = [
{ id: 1, name: "Rahul", age: 20 },
{ id: 2, name: "Amit", age: 21 }
];
```

```
// GET All Students
app.get("/students", (req, res) => {
res.json(students);
});

// GET Single Student
app.get("/students/:id", (req, res) => {
const student = students.find(
s => s.id === parseInt(req.params.id)
);

if (!student) {
return res.status(404).json({ message: "Student Not Found" });
}

res.json(student);
});
```

```
// POST Student
app.post("/students", (req, res) => {

const student = {
id: students.length + 1,
name: req.body.name,
age: req.body.age
};

students.push(student);

res.status(201).json(student);

});
```

```
// PUT Student
app.put("/students/:id", (req, res) => {

const student = students.find(
s => s.id === parseInt(req.params.id)
);

if (!student) {
return res.status(404).json({ message: "Student Not Found" });
}

student.name = req.body.name;
student.age = req.body.age;

res.json(student);

});
```

```
// PUT Student
app.put("/students/:id", (req, res) => {

const student = students.find(
s => s.id === parseInt(req.params.id)
);

if (!student) {
return res.status(404).json({ message: "Student Not Found" });
}

student.name = req.body.name;
student.age = req.body.age;

res.json(student);

});
```

```
// DELETE Student
app.delete("/students/:id", (req, res) => {

const index = students.findIndex(
s => s.id === parseInt(req.params.id)
);

if (index === -1) {
return res.status(404).json({ message: "Student Not Found" });
}

students.splice(index, 1);

res.json({
message: "Student Deleted"
});

});
```